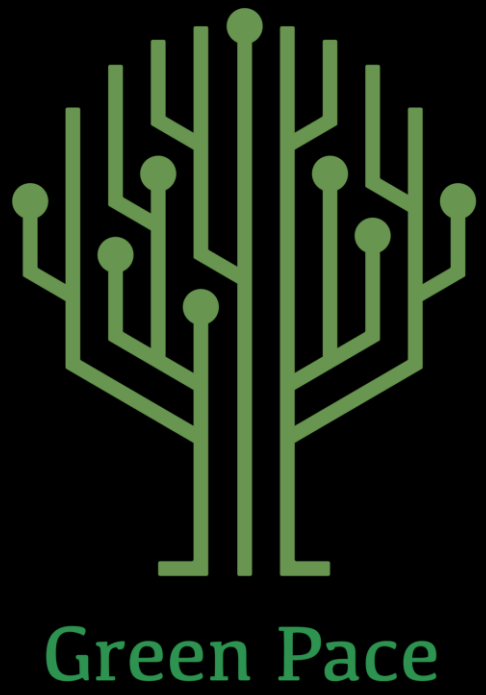
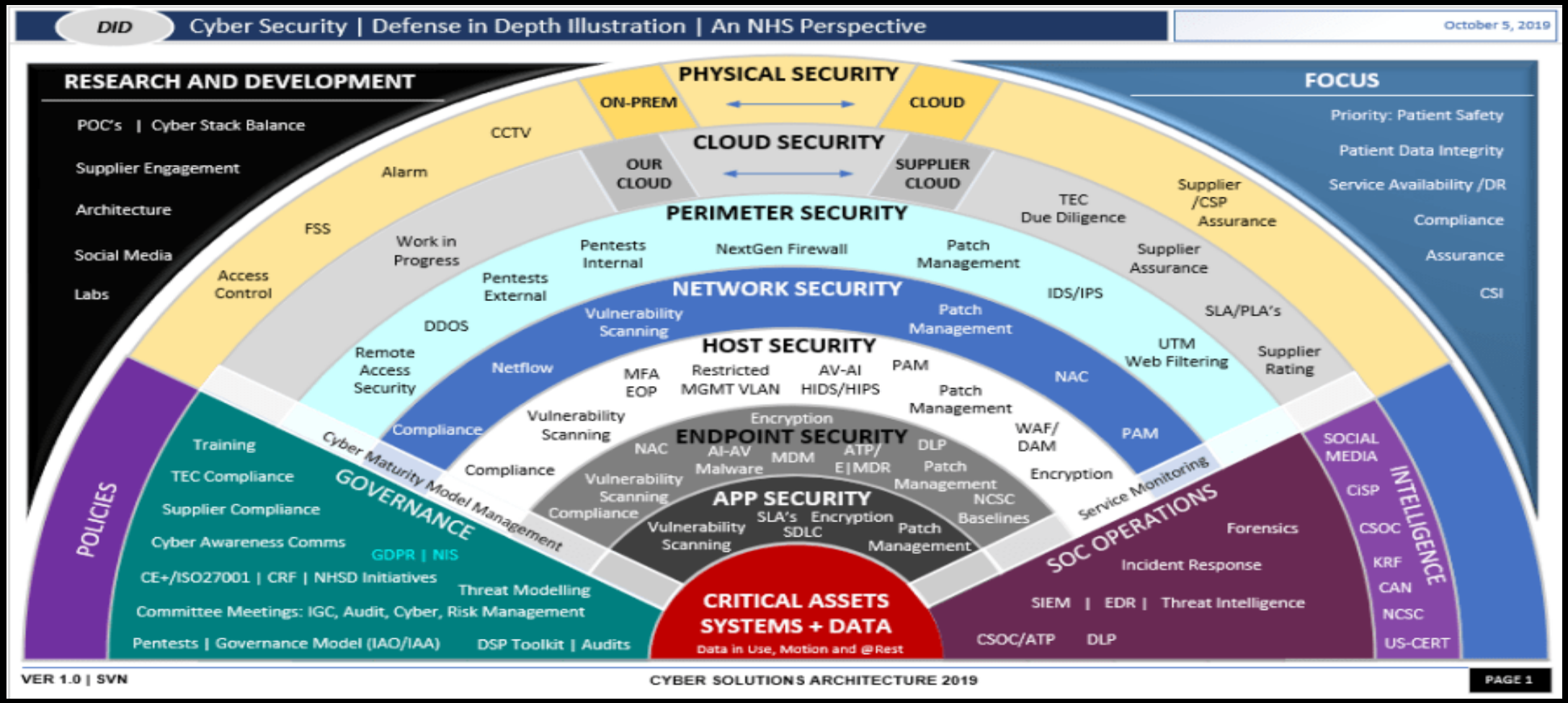
Green Pace



Security Policy Presentation
Developer: *Justine Kummer*

OVERVIEW: DEFENSE IN DEPTH



Green Pace

Defense-In-Depth

This security policy works to shield the software from multiple aspects. Using defense in depth, attacks can be prevented from multiple fronts and areas. This way of malware is through physical security; there's still multiple areas it can be detected and stopped. This is why each level should be evaluated and implemented.

THREATS MATRIX

Likely STD-003-CPP STR50-CPP. Guarantee that storage for strings has sufficient space for character data and the null terminator STD-005-CPP MEM30-C. Do not access freed memory	Probable STD-001-CPP INT08-C. Verify that all integer values are in range STD-002-CPP EXP54-CPP. Do not access an object outside of its lifetime STD-007-CPP ERR51-CPP. Handle all exceptions STD-008-CPP MEM35-C. Allocate sufficient memory for an object STD-0010-CPP MSC41-C. Never hard code sensitive information
High STD-004-CPP STR02-C. Sanitize data passed to complex subsystems	Unlikely STD-006-CPP MSC11-C. Incorporate diagnostic tests using assertions STD-009-CPP FIO42-C. Close files when they are no longer needed

Green Pace

10 PRINCIPLES

Standard 1:	Principles:
1. Validate Input Data	INT08-C. , STR02-C
2: Heed Compiler Warnings	EXP54-CPP
3: Architect and Design for Security Policies	INT08-C, EXP54-CPP, MSC11-CPP, ERR51-CPP
4: Keep It Simple	MEM30-C, MEM35-C
5: Default Deny	MSC11-C, ERR51-CPP
6: Adhere to the Principle of Least Privilege	FIO42-C, MSC41-C
7: Sanitize Data Sent to Other Systems	STR02-C
8: Practice Defense in Depth	FIO42-C, MSC41-C
9: Use Effective Quality Assurance Techniques	EXP54-CPP, MEM30-C
10: Adopt a Secure Coding Standard	MEM35-C

Green Pace

CODING STANDARDS

Validate Input Data: Validating input data means examining and filtering data into a system. It's important for security because the inspection filters out data that may harm the program or be used by malicious actors, such as SQL injections or input that breaks set rules and may cause code malfunctions.

Heed Compiler Warnings: By heeding compiling warnings, you can make a stable code that avoids malfunctions and exploitable loopholes. These warnings are made to help coders keep up with the latest practices and avoid logic errors in the coding process. By heeding these warnings while coding, you can avoid major obstacles, and later issues. Such as a coding warning about old coding styles, which were upgraded to prevent known security flaws.

Architect and Design for Security Policies: By implementing security strategies from early on processes, you can avoid later issues of implementing security to finished sections. As well as lessen security patches later.

Keep It Simple: While keeping it simple may seem like an easy concept it requires well thought out structure. As you code, some sections may seem to require complex code solutions, while it works, it makes it harder for others to understand. As well as decreasing the reusability of the code and making it harder to audit. This increases the chance of unseen vulnerabilities, such as loopholes and hidden bugs.

Default Deny: As the name states, default deny restricts a user's access, unless a rule is met. An easy example is creating a password. All input is denied unless it meets a strict criteria such as no special symbols or being over 12 characters.

Adhere to the Principle of Least Privilege: The principle of least privilege is a very good way to help prevent attacks. By using this you can limit who has what privileges, as well as add specific security measures. Such as a user can browse a website but needs an account to interact. This prevents the mass populace from using it without some type of accountability and mitigates the damage one user can do, based off their tier.

Sanitize Data Sent to Other Systems: This is an important concept; it's about cleaning any outgoing data to other systems. This can protect systems from many issues, including command attacks. This prevents attacks by sending harmless data stripped of any possible malicious code.

Practice Defense in Depth: Practicing Defense in Depth is to have multiple defensive layers throughout multiple areas of an application. This way if a hacker breeches one layer, another blockage is in place to protect sensitive data, the user, and the program.

Use Effective Quality Assurance Techniques: Quality assurance techniques help to prevent defects before later stages. By checking and practicing quality techniques you can help ensure a high-quality reduction. This can be done by peer review, audits, and testing.

Adopt a Secure Coding Standard: Adopting a code standard is a well-thought-out process. By doing so, a team can work uniformly to meet a shared set of rules and instructions. Leading to team cohesion and coding. This can prevent mismatched standards and practices, leading to a more stable product throughout the SDLC life cycle.

Green Pace

ENCRYPTION POLICIES

Encryption at rest	Encryption at rest is how data is protected as it sits. This can range from your cloud account to your saved files on your phone. This practice protects your data when its not in use through encryption tools, and security systems in place.
Encryption in flight	This protects data on the move. Such as emails, files being sent and more. This can cover data within a given network or outside of one. Email encryption is an example, so that an email can't be intercepted and used by bad actors or malware. Firewalls are also used to read bad data coming in and catching it while warning the user. This should be taken into consideration early on.
Encryption in use	Encryption in use is the protection of data actively in use. Such as the computers RAM or data actively being edited. This data is trickier since its actively decrypted as the processor reads it. Leading to a possible vulnerability if not properly encrypted from possibilities such as memory-scraping malware.

Green Pace

TRIPLE-A POLICIES

Authentication	Authentication is used to validate a user. This can be done in multiple ways, such as login screens and MFA. This proves the user is who they say they are. Without these measures bad actors could easily breach an application with no way to verify them.
Authorization	Authorization is how much trust a user has. This can be used to limit the privileges a user has. One example is a trip booking website. Since only administrators can add or edit a trip. Basic users would not have this ability or a way to do so without administrator's assistance.
Accounting	Accounting is essential this way, each user is accountable and tracible. If a system is accessed, or changed, a log should be updated.

Green Pace



Unit Testing

“Unit Testing

Unit Testing is a software testing method in which individual units or components of a software application (such as functions, methods, or classes) are tested in isolation to verify that they work correctly as expected.

It helps find and fix defects at the very beginning of the development cycle, reducing the cost and effort of debugging later.

It promotes writing modular, clean, and maintainable code by ensuring each small part of the application behaves correctly on its own.”

(geeksforgeeks.org, 2024)

Green Pace

Removing one item decreases the size by one : positive test

```
//positive test
TEST_F(CollectionTest, removingOneItemDecreasesTheSizeByOne)
{
    add_entries(10); //add 10 values to the collection
    size_t sizeOfCollection = collection->size(); //size is stored in SizeOfCollection
    collection->pop_back();
    EXPECT_EQ( collection->size(), sizeOfCollection - 1); // asserts the size decreased by 1
}
```

This test is done to verify the size change of the collection. It prevents subtle bugs and logic errors.

Removing one item decreases the size by one : positive test

```
//positive test
TEST_F(CollectionTest, removingOneItemDecreasesTheSizeByOne)
{
    add_entries(10); //add 10 values to the collection
    size_t sizeOfCollection = collection->size(); //size is stored in SizeOfCollectioN
    collection->pop_back();
    EXPECT_EQ( collection->size(), sizeOfCollection - 1); // asserts the size decreased by 1
}
```

```
[ RUN      ] CollectionTest.removingOneItemDecreasesTheSizeByOne
[       OK ] CollectionTest.removingOneItemDecreasesTheSizeByOne (0 ms)
```

Always Fail : positive test

```
TEST_F(CollectionTest, AlwaysFail)
{
    FAIL();
}
```

This test serves as evidence that the testing framework is running correctly. It also proves the validity of the testing.

Always Fail : positive test

```
TEST_F(CollectionTest, AlwaysFail)
{
    FAIL();
}
```

```
[ RUN      ] CollectionTest.AlwaysFail
C:\Users\justi\Desktop\snhu\GoogleTest\GoogleTest\pch.cpp(87): error: Failed
[  FAILED  ] CollectionTest.AlwaysFail (0 ms)
[ RUN      ] CollectionTest.CanAddToEmptyVector
```

negative test: if collection is empty throw an error

```
//negative test
 TEST_F(CollectionTest, ifCollectionIsEmptyThrowAnError)//name is what the test runs
 {
     EXPECT_THROW(collection->at(0), std::out_of_range); //if the collection is empty, expect an error
 }
```

Checks to make sure collection is empty and throws a out of range error. As a negative test, an error is the expected outcome. To pass the test must throw the out-of-range error, if it doesn't. the test fails

negative test: if collection is empty throw an error

```
//negative test
TEST_F(CollectionTest, ifCollectionIsEmptyThrowAnError)//name is what the test runs
{
    EXPECT_THROW(collection->at(0), std::out_of_range); //if the collection is empty, expect an error
}
```

```
[ RUN      ] CollectionTest.ifCollectionIsEmptyThrowAnError
[       OK ] CollectionTest.ifCollectionIsEmptyThrowAnError (0 ms)
```

Verify Clear Erases The Collection

```
TEST_F(CollectionTest, verifyClearErasesTheCollection)
{

    add_entries(5); //add 5 values to the collection
    ASSERT_EQ(collection->size(), 5); //the collection size is five

    collection->erase(collection->begin(), collection->end()); //erase the collection
    EXPECT_EQ(collection->size(), 0); //it now contains 0 values
    EXPECT_TRUE(collection->empty()); //true if the collection is empty or no collection
}
```

Adds 5 values to the collection, then erases the collection. Then it checks the collection size properties. It fails the test if collection size is not zero.

Verify Clear Erases The Collection

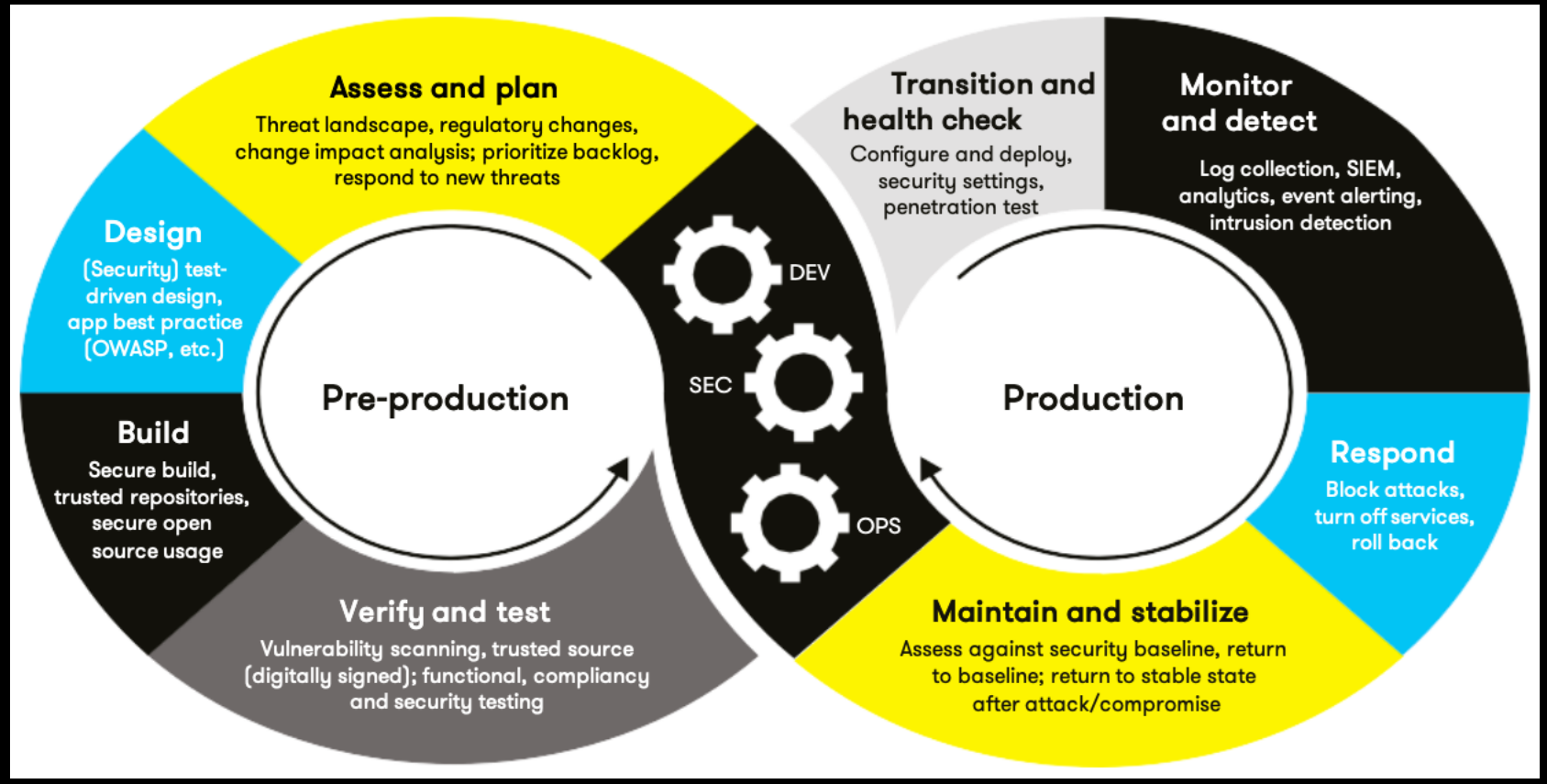
```
TEST_F(CollectionTest, verifyClearErasesTheCollection)
{

    add_entries(5); //add 5 values to the collection
    ASSERT_EQ(collection->size(), 5); //the collection size is five

    collection->erase(collection->begin(), collection->end()); //erase the collection
    EXPECT_EQ(collection->size(), 0); //it now contains 0 values
    EXPECT_TRUE(collection->empty()); //true if the collection is empty or no collection
}
```

```
[ RUN      ] CollectionTest.verifyClearErasesTheCollection
[       OK ] CollectionTest.verifyClearErasesTheCollection (0 ms)
[ RUN      ] CollectionTest.verifyErasesTheCollection
[       OK ] CollectionTest.verifyErasesTheCollection (1 ms)
```

AUTOMATION SUMMARY



Green Pace

TOOLS

Automation is an important tool throughout each process. A variety of tools help not only amplify your progress but expose any vulnerabilities throughout the process in a cost-effective way. Using the tools provided security flaws can be found quicky as the developer works. Automation can help enforce any standards given. This diagram shows how automation can be used in each step of the pre-production and production fields. Even accounting for future attacks in the post-production stage and maintaining the project throughout post-production stages.

Green Pace

RISKS AND BENEFITS

Security is an important piece of a programs production at all levels. Leaving issues for later can leave vulnerabilities to compound and build. This not only leaves the chance of it passing unnoticed, but it also makes it harder to fix. Fixing vulnerabilities early on, fixes issue and logic issues while their fresh. This helps keep cost low and issues minimal.

Green Pace

RECOMMENDATIONS

Pre-production is well thought out. Showing how the teams assess and plan before designing or building. Considering security throughout the process. As well as verifying and testing. In this stage tools like CPPcheck will be crucial, since tools like this can do an exhaustive check for multiple vulnerabilities. A security audit would also be a good step, testing for common attacks.

Green Pace

CONCLUSIONS

My conclusion is that by adopting a security process that follows these standards, we can optimize out security framework. Using defense-In-Depth to protect our software from malicious actors. As well as using workers, alongside out automation goals. Brining up their productivity and output. With a few recommendations added, as well. Such as adding code audits after each stage to check for logic breaks and uncommon vulnerabilities. Using higher than standard tools, like CPPCheck and Coverity for static analysis testing. As well as dynamic testing, such as injecting malicious data and seeing how our defenses hold and where needs more attention.

Green Pace

REFERENCES

- **GeeksforGeeks. (n.d.). *Unit testing – software testing*. https://www.geeksforgeeks.org/software-testing/unit-testing-software-testing/**
- Carnegie Mellon University Software Engineering Institute. (n.d.). Secure coding standards. https://cmu-sei.github.io/secure-coding-standards/



Green Pace